Summative Assignment 1: AR Simulation Proposal, Requirements & Software Design

Fatiro AR - Experimental Housing Structures Simulator

Author: NTARE GAMA Allan · Oct 15, 2026

Fatiro AR is a mobile augmented-reality simulation. In it, construction and engineering learners build experimental house forms at tabletop scale, then load-test them under Rwandan site conditions. Learners place a hillside plot on a real table and assemble a structure member by member in the correct construction sequence. They then apply wind, rain, occupancy and earthquake loads and watch the structure respond. The structures are new forms such as hexagonal hive clusters, saddle-shell pavilions and rain-funnel ring houses, not the standard rectangular house.

The name comes from urufatiro, Kinyarwanda for "foundation": the app gives learners a foundation for building new forms safely.

This document covers Summative Assignment 1. It contains the project proposal, the Software Requirements Specification (SRS), the UML design, the proposed architecture, the OOP and design-pattern decisions and the data management plan. It is the baseline for Assignment 2 (technical implementation in Unity with AR Foundation).

Item	Value
Domain	Construction and engineering
Platform	Android phones with ARCore, built in Unity 2022 LTS + AR Foundation 5
Primary users	Construction and engineering learners, TVET trainees, instructors
Delivery target	Working MVP with 3 house designs (Assignment 2)

1. Project Proposal

Fatiro AR lets learners build experimental, never-before-built house forms at tabletop scale and test them under simulated loads. Mistakes happen on a phone screen instead of on a construction site.

1.1 Problem

Construction learners in Rwanda mostly learn structural behaviour from 2D drawings, formulas and a narrow set of conventional rectangular house designs. They rarely see how unconventional forms carry load. Examples are shells, hexagonal clusters and spiral terraces. Physical prototypes are expensive and slow, and full-scale mistakes are dangerous.

Rwanda's conditions raise the cost of these gaps:

- **Steep terrain.** Many plots sit on slopes that need stepped or raised foundations.
- **Intense rainy seasons.** The floods and landslides of May 2023 in the Western and Northern Provinces showed how fast drainage and foundation weaknesses become fatal.
- **Seismic activity** along the Western Rift, especially around Lake Kivu.

The result is that new house forms are seldom attempted. When they are, errors in load paths or construction order are found on site, where they are costly to fix.

1.2 Target users

User	Need	Use of Fatiro AR
Civil engineering and architecture students (university level)	Build intuition for how new structural forms carry load	Assemble designs, run load tests, compare materials and bracing
TVET construction trainees and site technicians (Rwanda Polytechnic / IPRC level)	Learn safe construction sequence before working on site	Guided Build mode with step-by-step stage validation
Instructors and lecturers	Set exercises and assess learners objectively	Load scenario files and review exported session reports

1.3 Proposed solution

Fatiro AR is an Android app that anchors a scaled hillside site on any flat surface. The learner chooses an experimental house design and site conditions (slope, soil, climate). They then assemble the structure in the correct sequence: foundation, primary structure, bracing, roof, envelope. A simplified structural solver applies load cases and colours every member by how close it is to failure. Members that exceed capacity fail visibly and can bring dependent members down with them. The learner reinforces the design (adds bracing, changes material) and re-tests. A session score and report record what they did.

1.4 Why AR

- **Spatial understanding.** Learners walk around a curved shell or a hexagonal cluster and look underneath it. A flat screen or drawing hides depth, curvature and connection details.
- **Physical context.** The model sits on a real table at a chosen scale (1:100 to 1:20), so several learners can gather around one model, as they would around a physical maquette.

- **Safe failure.** Collapse, flooding and earthquake damage can be shown in place without risk or material cost. Physical models cannot do this repeatedly.
- **Accessible hardware.** It runs on ARCore phones that learners and institutions already own. No headset or lab is needed.
- **Better than plain 3D.** A 3D viewer shows the result. AR places the learner's hands and body in the assembly task, which is the skill being trained.

1.5 Experimental design catalogue

The catalogue contains six original forms designed for steep, wet, seismic sites. Each has a Kinyarwanda name that describes its form. Three are delivered in the MVP. The other three are defined as data and are planned for later releases.

ID	Design	Concept	Key engineering lesson	MVP
HX-01	Umutiba ("beehive")	Interlocking hexagonal pods on stepped stilts that climb the slope and share walls	Load sharing between cells; stilt bracing on slopes	Yes
HP-02	Umuheto ("bow")	Hyperbolic-paraboloid (saddle) roof made of straight bamboo members, with two low corners that drain to cisterns	Doubly curved shells built from straight members; anchoring uplift	Yes
RF-04	Isoko ("spring, water source")	Ring of rooms around a courtyard under an inverted-cone roof that feeds a central cistern	Ring-beam action; water loading and drainage	Yes
SP-03	Amaterasi ("terraces")	Rooms on a spiral descending the hillside; each roof is the terrace of the room above	Retaining loads; stepped foundations	Later
CT-05	Inkingi ("pillar")	Stacked modules, each rotated 15°	Cantilevers and torsion	Later

ID	Design	Concept	Key engineering lesson	MVP
		and cantilevered from a central core		
GS-06	Umugano ("bamboo")	Double-curved lattice of bent bamboo	Form-active structures; buckling	Later

1.6 Simulation scenario

A TVET trainee is set to the exercise "Build HX-01 Umutiba on a 20° laterite slope and survive a shaking level 3 earthquake and an 80 mm/h storm".

1. The trainee scans a table. The app detects the surface, and the trainee taps to place the sloped plot.
2. The trainee chooses HX-01 Umutiba, a 20° slope, laterite soil and the Guided Build mode.
3. The trainee builds the stilt foundations, then the hexagonal frames, then the bracing, roof panels and walls. The app rejects a roof panel placed before its supporting ring beam and names the missing member.
4. The trainee runs the earthquake test. Two unbraced downhill stilts turn red and fail, and the pod above them drops.
5. The trainee adds cross-bracing to the downhill stilts, changes them from timber to steel and re-runs the test. Peak utilisation falls from 128% to 86% (amber, no failure).
6. The trainee runs the storm. Roof runoff is routed to drains and the collected volume is shown.
7. The session ends with a score (assembly accuracy, safety, material cost). The trainee exports the report to the instructor.

1.7 Core interactions

- Scan a surface and tap to place the site; pinch to scale, twist to rotate, drag to move it.
- Drag structural members from a tray and release them near a connection node to snap them in place.
- Choose a material for a member and add or remove bracing.
- Set load sliders (wind, rain, shaking level) and run the test.
- Tap a member to inspect its material, load and utilisation.
- Undo, pause, reset and end the session.

1.8 Expected outcome

- Learners can explain the load path of at least three unconventional house forms.
- Learners apply the correct construction sequence, which the app measures as the share of placements accepted on the first try.
- Learners reduce member failures by reinforcing a design and re-testing it.
- Instructors receive comparable, exportable evidence of each learner's performance.

1.9 Scope

In scope (MVP)	Out of scope
Android ARCore phones, offline use	iOS, head-mounted displays, multi-user shared AR sessions
3 designs (Umutiba, Umuheto, Isoko), 5 materials, 5 load cases	Certified structural analysis or code compliance (e.g. Eurocode)
Guided Build, Free Build and Load Test modes	Designing new house forms from scratch inside the app
Local learner profiles, session history, JSON/CSV export	Cloud accounts, online leaderboards, LMS integration
Instructor scenario files (JSON)	Cost estimates based on live market prices

2. Software Requirements Specification

2.1 Introduction

Purpose. This SRS defines the functional and quality requirements of Fatiro AR version 1.0 (MVP). It is the reference for implementation, testing and assessment in Assignment 2.

Problem statement. Construction learners have no safe, affordable way to assemble unconventional house structures and see how they behave under the slope, rain and seismic conditions found in Rwanda. As a result, new forms are rarely attempted, and sequencing and load-path errors are discovered on site.

Target users.

- *Learner:* a university or TVET student, or a site technician. Basic smartphone skills; no prior AR experience is assumed.
- *Instructor:* sets scenarios and reviews reports. Has the same app access as a learner, plus the ability to load scenario files.

Objectives.

1. Let learners assemble at least 3 experimental house designs in AR, in the correct construction sequence.
2. Simulate 5 load cases (dead, live, wind, rain, seismic) and show per-member utilisation and failure.
3. Let learners modify a structure and compare results before and after the change.
4. Score each session and store a report that can be exported.
5. Run fully offline on mid-range Android phones.

Scope. Section 1.9 defines what is in and out of scope. The system is an educational simulator. Its structural results are simplified and must not be used for real construction decisions.

Assumptions.

- Users have an ARCore-supported Android phone (Android 10 or later).
- Sessions take place indoors on a flat, textured surface (table or floor) in normal lighting.
- Member capacities and loads are simplified, educational values. Instructors accept a ±15% deviation from hand calculations.
- One learner uses one device at a time.

Constraints.

- Built with Unity 2022 LTS, AR Foundation 5 and C#. Delivered within the Assignment 2 timeline by a single developer.
- No dependency on a network connection or a backend server.
- Mid-range phone GPU and thermal limits restrict model detail to about 400 members and about 150k triangles per scene.
- Learner data is limited to a display name. No other personal data is collected.

2.2 Functional requirements

The 28 requirements below cover the whole simulation lifecycle, from AR setup through assembly, load testing and scoring to data. Each requirement can be verified on its own by a single test. Priority uses MoSCoW: M = Must, S = Should.

ID	Lifecycle stage	Requirement	Priority
FR-01	AR setup	The system shall request camera permission on first launch. If permission is denied, the system shall show a screen that explains why the camera is required.	M
FR-02	AR setup	The system shall detect horizontal planes using AR Foundation and display each detected plane with a grid overlay.	M
FR-03	AR setup	The user shall be able to place the site plot on a detected horizontal plane by tapping it.	M
FR-04	AR setup	The user shall be able to move, rotate and	M

ID	Lifecycle stage	Requirement	Priority
		scale the placed site, with the scale limited to between 1:100 and 1:20.	
FR-05	AR setup	When AR tracking is limited or lost for more than 3 s, the system shall pause the simulation and show a "move the device slowly" prompt.	M
FR-06	Setup	The system shall display a design catalogue that lists each available design with its name, thumbnail, description and difficulty level.	M
FR-07	Setup	The user shall be able to set the site conditions before building: slope (0°, 10°, 20° or 30°), soil type (rock, laterite or clay) and climate preset.	M
FR-08	Setup	The user shall be able to choose Guided Build, Free Build or Load Test (prebuilt structure) mode.	M
FR-09	Assembly	In Guided Build mode, the system shall present the construction stages in a fixed order (foundation, primary structure, bracing, roof, envelope) and show the next required member as a ghost outline.	M

ID	Lifecycle stage	Requirement	Priority
FR-10	Assembly	The system shall snap a dragged member to a valid connection node when it is released within 5 cm (real-world distance) of that node. Otherwise, the system shall return the member to the tray.	M
FR-11	Assembly	The system shall reject a member placed before its prerequisite members, leave the member unplaced and show a message naming the missing prerequisite.	M
FR-12	Assembly	The user shall be able to assign each member a material from the list the design allows (bamboo, timber, steel, reinforced concrete or compressed-earth block).	M
FR-13	Assembly	The user shall be able to undo and redo the last 20 build actions (placement, removal, material change).	S
FR-14	Assembly	The system shall mark a build stage as complete when all of its required members are placed and shall then unlock the next stage.	M
FR-15	Load test	The user shall be able to configure and run each of 5 load cases:	M

ID	Lifecycle stage	Requirement	Priority
		dead load, occupancy live load, wind (0–150 km/h), rainfall (0–100 mm/h) and ground shaking (levels 0–5).	
FR-16	Load test	For each load test, the system shall compute a utilisation ratio (applied load ÷ capacity) for every placed member.	M
FR-17	Load test	The system shall colour every member by utilisation: green below 70%, amber from 70% to 100%, red above 100%.	M
FR-18	Load test	The system shall mark every member above 100% utilisation as failed, play its failure animation and also fail every member that depends only on failed supports.	M
FR-19	Load test	In a rainfall test, the system shall animate water flowing from the roof to the drainage points and report the runoff collected in litres per hour.	S
FR-20	Inspection	When the user taps a member, the system shall show its name, build stage, material, current load and utilisation percentage.	M
FR-21	Iteration	After a load test, the user shall be able to	M

ID	Lifecycle stage	Requirement	Priority
		modify the structure, re-run the same load case and see the before and after peak utilisation side by side.	
FR-22	Assessment	When a session ends, the system shall calculate and display a score out of 100: assembly accuracy 40%, structural safety 40%, material cost efficiency 20%.	M
FR-23	Data	The system shall record each session: learner ID, design ID and version, site conditions, every placement attempt (timestamp and result), every load test with its results, and the final score.	M
FR-24	Data	The user shall be able to view a list of past sessions (date, design, score) and open the details of any of them.	S
FR-25	Data	The user shall be able to export a session report as a JSON or CSV file through the Android share sheet.	M
FR-26	Instructor	An instructor shall be able to import a scenario file (JSON) that sets the design, site conditions, required load cases and pass score. Sessions	S

ID	Lifecycle stage	Requirement	Priority
		started from that scenario shall use those settings.	
FR-27	Feedback	The system shall play a confirmation sound and a 50 ms vibration when a member snaps, a different sound when a placement is rejected, and allow audio to be muted in settings.	S
FR-28	Profile	The user shall be able to create, select and delete a local learner profile identified by a display name.	M

2.3 Non-functional requirements

Each quality requirement has a measurable target and a verification method. Unless stated otherwise, the reference device is a mid-range ARCore phone with 6 GB RAM (e.g. Samsung Galaxy A54).

ID	Quality	Requirement	Verification
NFR-01	Performance	The app shall render at 30 fps or more on the reference device with a complete design of up to 400 members placed.	Unity Profiler, 5-minute capture
NFR-02	Performance	A load test on a structure of up to 400 members shall return results within 2 s.	Timed EditMode test and on-device log
NFR-03	Performance	A cold start shall reach the AR-ready screen within 8 s. The first plane shall be detected within 10 s on a textured surface under	Stopwatch test, 10 trials

ID	Quality	Requirement	Verification
		indoor lighting of at least 300 lux.	
NFR-04	Usability	At least 80% of first-time users shall place the site and complete the first Guided Build stage within 5 minutes without help.	Usability test, 5 or more participants
NFR-05	Usability	Touch targets shall be at least 48 dp. Body text shall be at least 14 sp. Every screen shall be reachable within 3 taps of the main menu.	UI review checklist
NFR-06	Accessibility	Utilisation shall be shown by colour plus a pattern and a numeric label, so it can be read in colour-blind simulation (protanopia, deuteranopia). Every audio cue shall have a visual equivalent.	Colour-blind filter review
NFR-07	Accessibility	All UI text shall be stored in localisation tables, with English and Kinyarwanda supplied.	String audit: no hard-coded UI text
NFR-08	Reliability	The session shall autosave after every completed stage and every load test. After a forced close, at most one stage of progress shall be lost.	Kill-process test
NFR-09	Reliability	The app shall complete 30-minute sessions without crashing in at	Soak test, 20 sessions

ID	Quality	Requirement	Verification
		least 99% of test sessions.	
NFR-10	Availability	Every feature except sharing an export shall work with no network connection.	Airplane-mode test
NFR-11	Compatibility	The app shall run on ARCore-supported devices with Android 10 (API 29) or later, in portrait and landscape. The install size shall be 250 MB or less.	Device matrix on 3 or more phones
NFR-12	Maintainability	A new house design shall be addable through data alone (prefab plus design asset), without changing C# code.	Add a test design in a code-freeze build
NFR-13	Maintainability	The solver, sequence validator and scoring logic shall have at least 70% line coverage from automated unit tests.	Unity Test Framework coverage report
NFR-14	Validity	For each MVP design, solver results shall be within ±15% of hand-calculated reference values for 3 benchmark load cases.	Benchmark test suite
NFR-15	Privacy	The app shall store no personal data beyond a display name, keep all data on the device and share data only when the user exports it.	Data-flow review
NFR-16	Efficiency	A 30-minute session shall use no more than	Battery stats test

ID	Quality	Requirement	Verification
		15% of the battery on the reference device.	

3. UML Design

Five UML diagrams model Fatiro AR. The use case and class diagrams are mandatory. The activity, sequence and state machine diagrams describe the system's behaviour. Use case IDs (UC-xx) match the traceability matrix in section 7.

3.1 Use case diagram

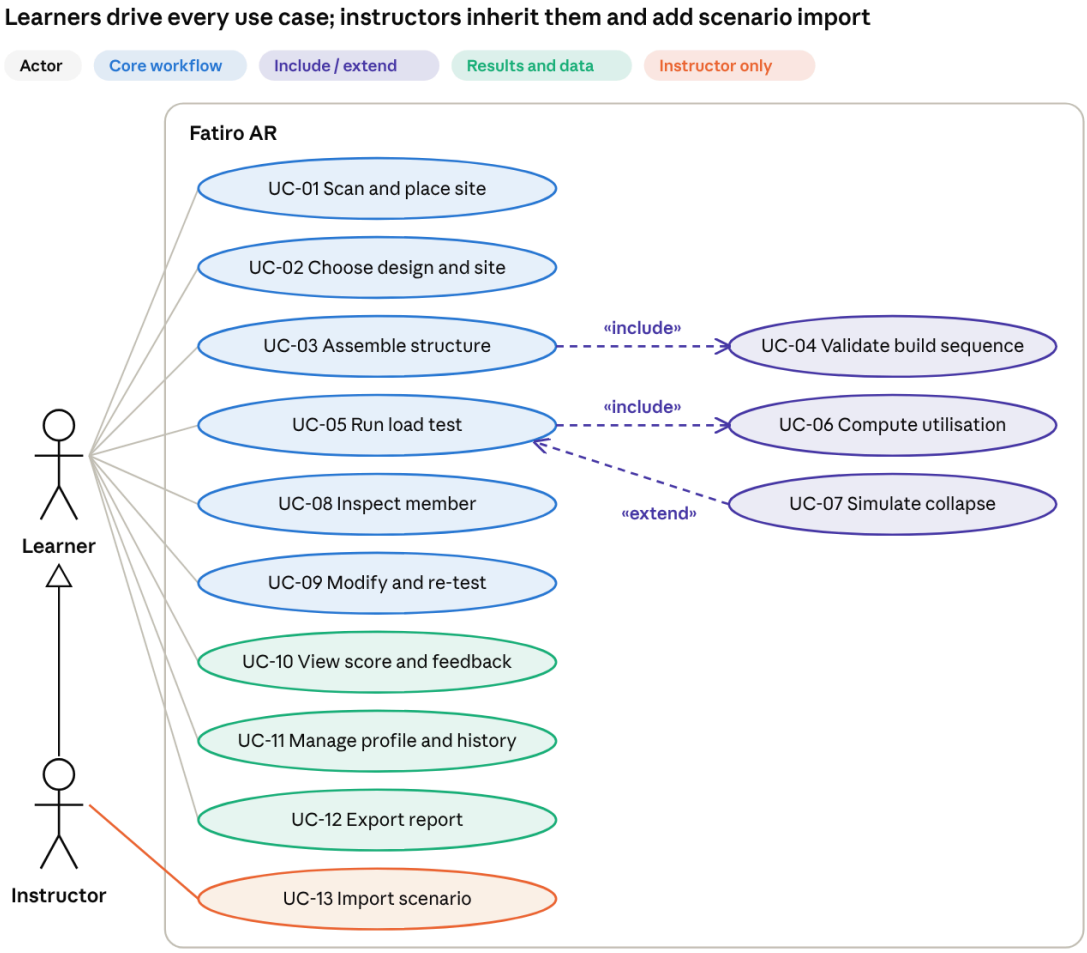


Figure 1 · Use case diagram · 2 actors, 13 use cases

The Instructor generalises the Learner, so the Instructor can perform every learner use case and can also import scenarios. Building a structure always includes sequence validation (UC-04), and every load test includes the utilisation calculation (UC-06). Collapse (UC-07) extends a load test only when a member exceeds 100% utilisation.

3.2 Class diagram

The class diagram is split into two views so it stays readable. Figure 2a shows the managers and services. Figure 2b shows the structural domain and data model. Abstract classes and interfaces carry the inheritance and polymorphism described in section 5.

Figure 2a · Managers and services

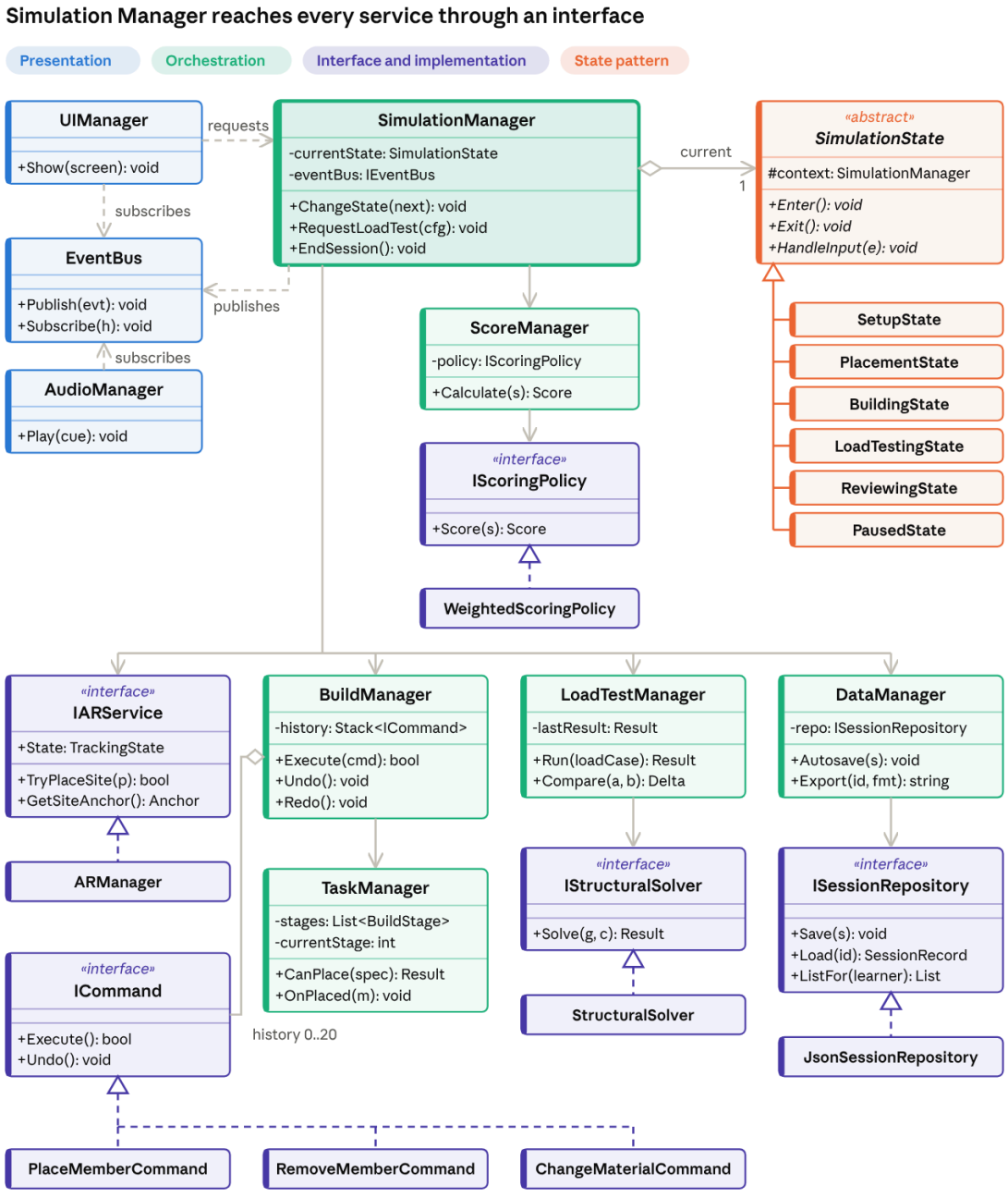


Figure 2a · Managers and services

Figure 2b · Structural domain and data model

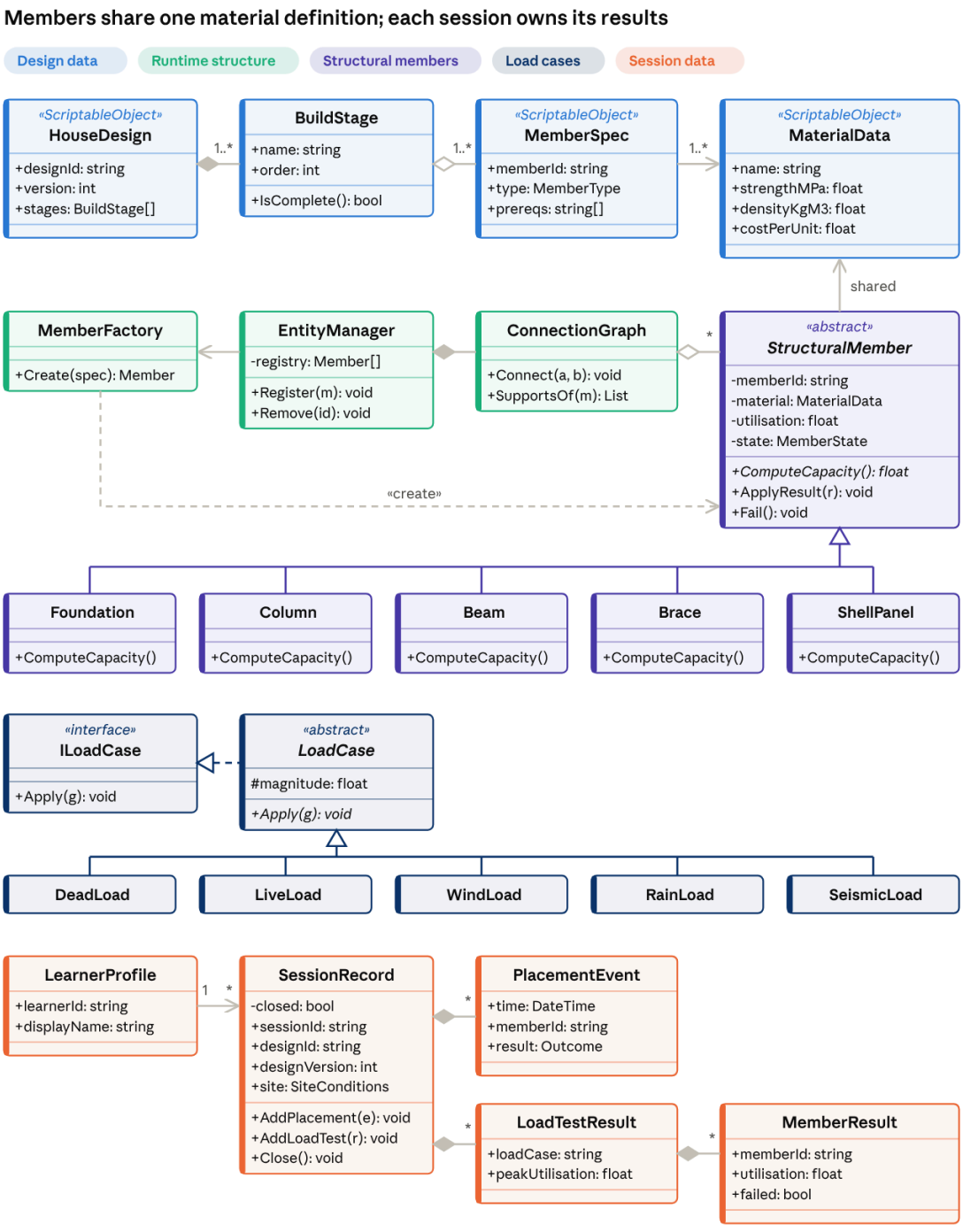


Figure 2b · Structural domain and data model

3.3 Activity diagram: a complete training session (Figure 3)

The session flows through setup, a validated build loop and an iterative test-and-reinforce loop. Every stage completion and every load test triggers an autosave.

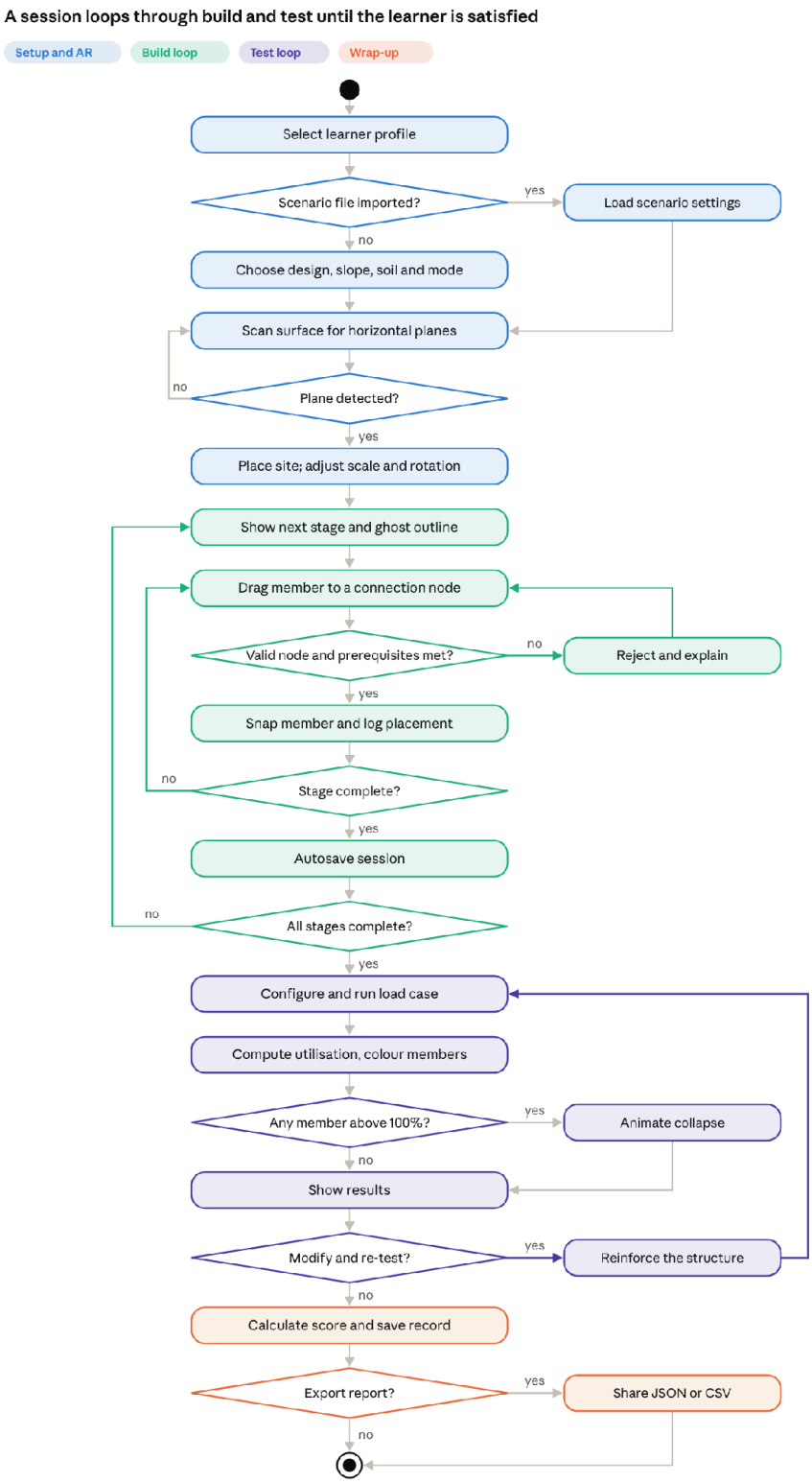


Figure 3 · Activity diagram · one training session

3.4 Sequence diagram: running a load test (Figure 4)

This sequence implements FR-15 to FR-19 and shows commands going down through interfaces and results coming back up as events.

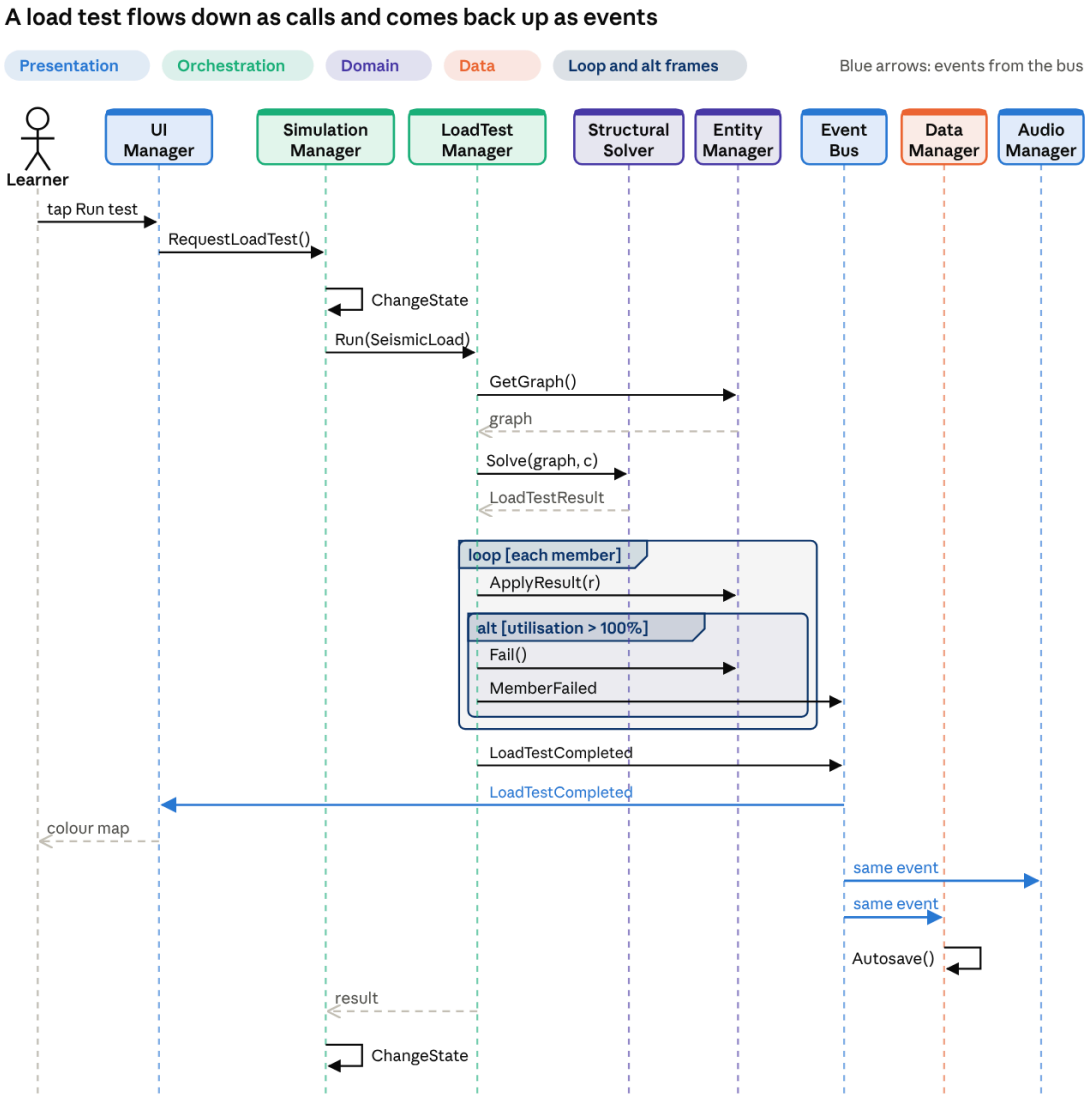


Figure 4 · Sequence diagram · running a load test

Solid arrows are synchronous calls and dashed arrows are returns. The first ChangeState moves the session to LoadTesting and the last to Reviewing. Fail() also fails every member that depends only on failed supports (FR-18).

3.5 State machine diagram: SimulationManager (Figure 5)

The Simulation Manager is always in exactly one state. Losing AR tracking from any active state moves it to Paused, and it returns to the previous state when tracking recovers (FR-05).

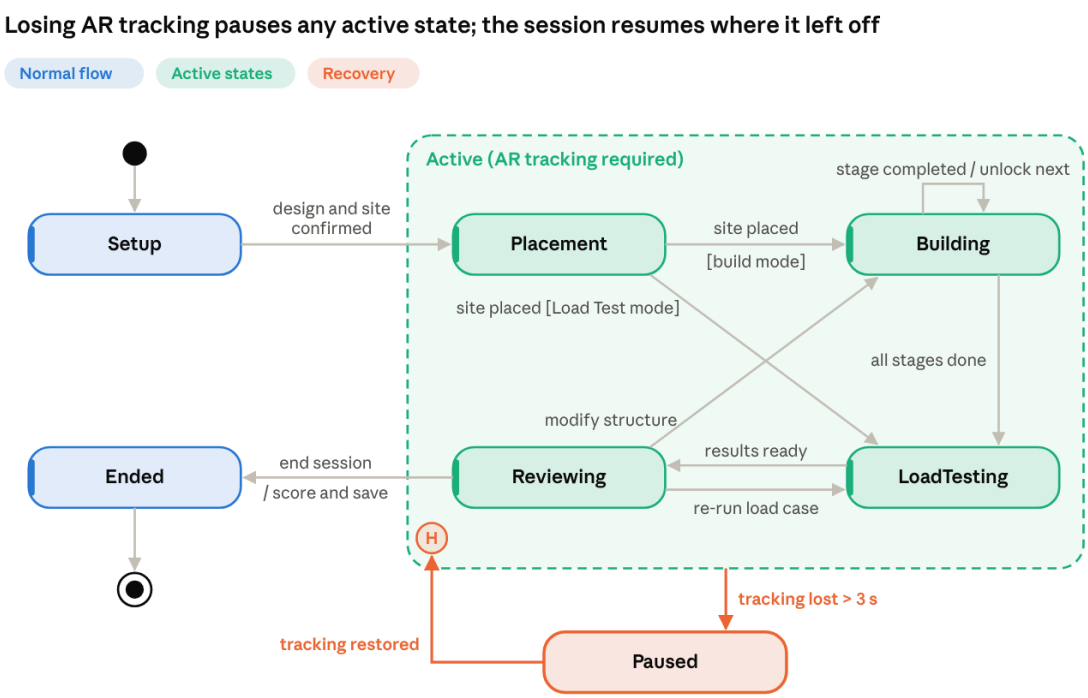


Figure 5 · State machine · SimulationManager

Placement, Building, LoadTesting and Reviewing sit inside the composite state Active. The shallow-history pseudostate (H) returns the session to whichever of these states it was in before tracking was lost.

4. Proposed Architecture

Fatiro AR uses a four-layer architecture. A Simulation Manager orchestrates the work, and an event bus decouples the layers. Commands flow down through interfaces, and notifications flow up as events, so no lower layer references a higher one.

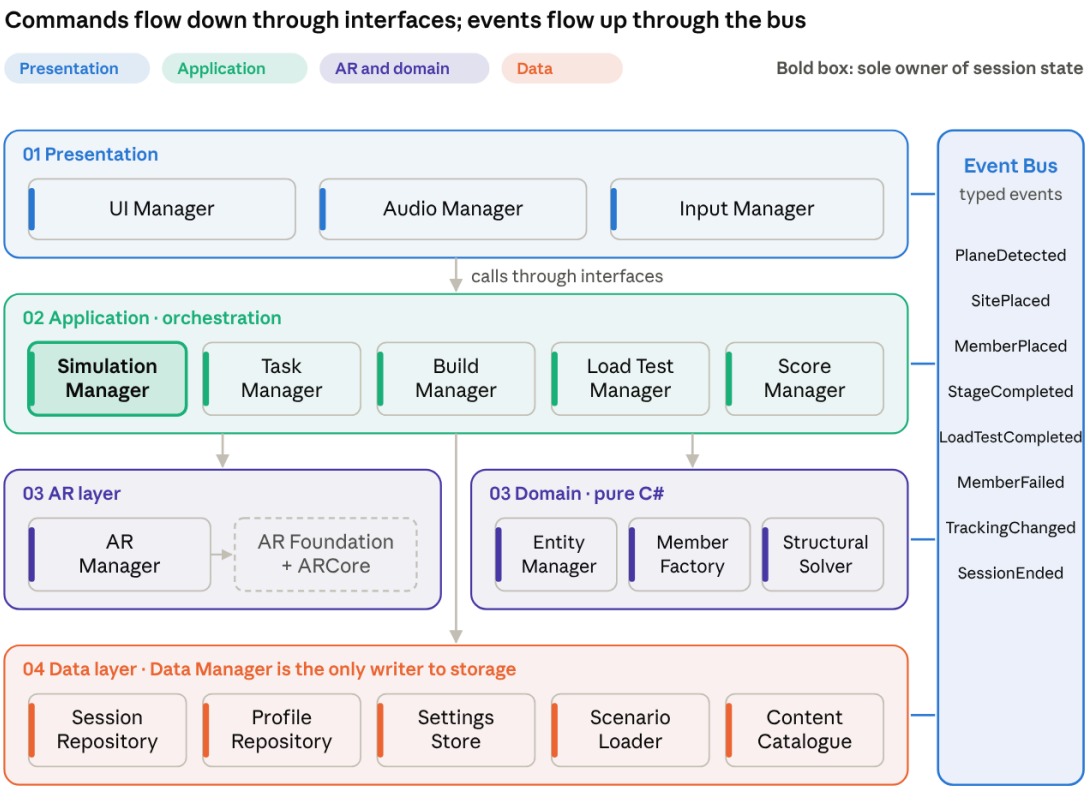


Figure 6 · Layered architecture · 5 layers and the event bus

The Application layer calls the AR, Domain and Data layers through interfaces. Every layer can publish to the Event Bus, and listeners such as the UI, Audio and Data managers react to those events without the publishers knowing about them.

4.1 Major systems and responsibilities

Layer	System	Responsibility	Key requirements
Presentation	UI Manager	Shows screens (catalogue, setup, build HUD, member inspector, results, history) and turns taps into requests to the Simulation Manager	FR-06, FR-20, FR-24
Presentation	Audio and Feedback Manager	Plays sound cues and haptics, and applies the mute setting	FR-27

Layer	System	Responsibility	Key requirements
Presentation	Input Manager	Converts raw touches into gestures (tap, drag, pinch, twist)	FR-03, FR-04, FR-10
Application	Simulation Manager	Owns the session state machine (Setup, Placement, Building, LoadTesting, Reviewing, Paused) and is the only system that starts transitions	FR-05, FR-08
Application	Task Manager	Holds the build stages, checks prerequisites and completes stages	FR-09, FR-11, FR-14
Application	Build Manager	Places and removes members and changes materials, through commands with an undo/redo history	FR-10, FR-12, FR-13
Application	Load Test Manager	Builds load cases, runs the Structural Solver, applies results and starts failure propagation	FR-15 to FR-19, FR-21
Application	Score Manager	Calculates the weighted session score	FR-22
AR	AR Manager	Wraps AR Foundation (ARSession, plane manager, raycast manager, anchors), reports tracking state and owns the site anchor	FR-02 to FR-05
Domain	Entity Manager	Creates members through the Member Factory, keeps the registry of placed members and the connection graph	FR-10, FR-18

Layer	System	Responsibility	Key requirements
Domain	Structural Solver	Pure C# with no Unity calls: distributes loads through the connection graph and returns a utilisation value per member	FR-16, NFR-02, NFR-14
Data	Data Manager	Session repository (JSON files), profile store, settings store, scenario loader and content catalogue	FR-23 to FR-26, FR-28

4.2 How the systems communicate

1. **Commands downwards.** The UI Manager calls `ISimulationController` methods, such as `RequestLoadTest(LoadCaseConfig)`. The Simulation Manager calls the application services through their interfaces (`IBuildService`, `ILoadTestService`, `IARService`, `ISessionRepository`).
2. **Events upwards.** Services publish typed events on the `EventBus`. The UI, Audio and Data managers subscribe to these events, so a publisher never knows who is listening.
3. **Dependency injection.** A single `AppBootstrap` composition root creates the services and passes their interfaces to the constructors that need them. No global singletons are used.
4. **Data access** goes only through the Data Manager's repositories. No other system reads or writes files.

Event	Published by	Consumed by
`PlaneDetected`, `TrackingChanged`	AR Manager	Simulation Manager, UI Manager
`SitePlaced`	AR Manager	Simulation Manager, Entity Manager
`MemberPlaced`, `PlacementRejected`	Build Manager	Task Manager, UI, Audio, Data
`StageCompleted`	Task Manager	Simulation Manager, UI, Data (autosave)
`LoadTestCompleted`, `MemberFailed`	Load Test Manager	UI, Audio, Data, Score Manager
`SessionEnded`	Simulation Manager	Score Manager, Data Manager, UI

This split gives high cohesion, because each manager owns one concern. It also gives low coupling, because only interfaces and events cross layer boundaries. The solver and validator stay pure C#, so they can be unit-tested without a phone (NFR-13).

5. OOP and Design Patterns

The design uses the four OOP principles to keep the structural model correct and extensible. It uses eight patterns, each chosen to solve a specific problem in this simulation.

5.1 OOP principles

Principle	Application in Fatiro AR
Encapsulation	`StructuralMember` exposes `Utilisation` and `State` as read-only. Only `ApplyResult(MemberResult)` can change them, so the UI cannot fake a result. `SessionRecord` becomes immutable after `Close()`. `ARManager` hides every AR Foundation type behind `IARService`.
Abstraction	Systems depend on interfaces, not concrete classes: `IARService`, `IStructuralSolver`, `ILoadCase`, `ISessionRepository`, `ICommand`, `IScoringPolicy`. The Load Test Manager asks for results without knowing how the solver computes them.
Inheritance	The abstract class `StructuralMember` is specialised by `Foundation`, `Column`, `Beam`, `Brace` and `ShellPanel`. The abstract class `LoadCase` is specialised by `DeadLoad`, `LiveLoad`, `WindLoad`, `RainLoad` and `SeismicLoad`. The abstract class `SimulationState` is specialised by one class per session state.
Polymorphism	`member.ComputeCapacity(material)` is computed differently for each type: buckling for a `Column`, bending for a `Beam`, axial force for a `Brace` and membrane action for a `ShellPanel`. `loadCase.Apply(model)` and `state.HandleInput()` are always called through the base type.

5.2 Design patterns

Pattern	Where it is used	Problem it solves
State	`SimulationManager` with the `SimulationState` subclasses	Without it, every input handler would branch on the current mode in long if/else chains. With it, each state accepts only the inputs and transitions that are legal for it.
Observer (Event Bus)	`EventBus` with typed events (section 4.2)	The UI, audio and autosave must react to simulation events without the publishers holding references to them. This keeps coupling low and lets new listeners be added freely.
Command	`PlaceMemberCommand`, `RemoveMemberCommand`, `ChangeMaterialCommand`	Provides undo/redo (FR-13). Each command is also logged as a placement attempt in the session record (FR-23).
Strategy	`ILoadCase` implementations, and `IScoringPolicy` (instructor scenarios can change the score weights)	Lets load cases and scoring rules be swapped or added without changing the Load Test Manager or the Score Manager.
Factory Method	`MemberFactory.Create(MemberSpec)`	Creating a member means choosing the subclass, the prefab and the material, then registering the member in the graph. Doing this in one place prevents duplicated, inconsistent setup code.
Repository	`ISessionRepository`, `IProfileRepository` with JSON implementations	Keeps the file format and file paths out of the game logic. The storage could later move to SQLite or the cloud without other changes.
Flyweight (ScriptableObject data)	`MaterialData`, `MemberSpec`, `HouseDesign` assets shared by all instances	Hundreds of members share one material definition instead of copying it. This saves memory and makes NFR-12 (designs added as data) possible.

Pattern	Where it is used	Problem it solves
Object Pool	Ghost outlines, rain particles, failure debris	Avoids repeated instantiate and destroy calls during collapse and rain, which cause garbage-collection pauses that break the 30 fps target (NFR-01).

Singletons are deliberately avoided. The `AppBootstrap` composition root injects dependencies instead, which keeps the managers testable and their dependencies visible.

6. Data Management Plan

All data stays on the device, and the app needs no server. The app keeps four kinds of data apart (assets, configuration, user/session and simulation runtime data), and each kind has its own storage, owner and lifetime.

6.1 Data categories

Category	Examples	Source	Format and storage	Retention	Access
Assets	Member prefabs, house meshes, textures, audio clips, design thumbnails, localisation tables	Authored by the developer	Unity prefabs, FBX, PNG and OGG files packaged in the APK (Addressables groups)	Lifetime of the app version; replaced on update	Read-only: Entity Manager, UI Manager, Audio Manager
Configuration (built in)	`HouseDesign`, `BuildStage`, `MemberSpec`, `MaterialData`, load-case presets, score weights	Authored by the developer; values checked against hand calculations	ScriptableObject assets in the build, each with a `version` field	Lifetime of the app version	Read-only at runtime, through the content catalogue
Configuration (instructor)	Scenario files: design ID, site conditions, required load cases, pass score	Instructor	JSON in `persistentDataPath/scenarios/`, checked against a	Until the instructor deletes the file	Read: Simulation Manager. Write: import only

Category	Examples	Source	Format and storage	Retention	Access
			schema when imported		
User settings	Mute, language, colour-blind mode, last profile used	User	Unity PlayerPrefs (key-value)	Until the app data is cleared	Read and write: Data Manager (settings store)
User profile	`learnerId` (GUID), `displayName`, `createdAt`	User	`profiles.json`	Until the profile is deleted (FR-28), which also deletes its sessions	Read and write: profile repository
Session data	`SessionRecord` with placement attempts, load-test results and score	Generated by the simulation	One JSON file per session: `sessions/{learnerId}/{sessionId}.json`	Latest 50 sessions per learner or 180 days, whichever limit is reached first. Older files are purged at startup.	Write: Data Manager on events. Read: history screen and export
Simulation runtime state	Placed members, connection graph, current utilisation, undo stack	Generated in memory	C# objects; a snapshot is written to `autosave.json`	Snapshot written at each stage and load test, deleted when the session ends	Entity, Build and Load Test managers; restored after a crash (NFR-08)
Diagnostic log	Errors, timings, tracking loss	Generated by the app	Rolling text file of at most 1 MB	7 days	Developer only; never exported automatically

6.2 Structure and relationships

One `LearnerProfile` has many `SessionRecords`. Each `SessionRecord` refers to one `HouseDesign` by `designId` plus `designVersion`, so old reports stay valid after the content is updated. A session holds

many `PlacementEvents` and many `LoadTestResults`, and each result holds one `MemberResult` per member. The `SessionRecord` schema is:

```
{
  "schemaVersion": 1,
  "sessionId": "b1e4c9a2-7f0d-4c55-9a61-2f3e8d0c1a77",
  "learnerId": "5c0d2e11-8a3b-4e7f-b2c4-91d6e0f4a3b8",
  "scenarioId": "tvet-hx01-quake3",
  "design": { "designId": "HX-01", "designVersion": 3 },
  "site": { "slopeDeg": 20, "soil": "laterite", "climate": "western-highlands" },
  "startedAt": "2026-10-15T09:02:11Z",
  "endedAt": "2026-10-15T09:27:40Z",
  "placements": [
    { "t": "2026-10-15T09:04:02Z", "memberId": "HX01-STILT-03", "stage":
"foundation", "result": "accepted" },
    { "t": "2026-10-15T09:09:15Z", "memberId": "HX01-ROOF-01", "stage": "roof",
"result": "rejected", "reason": "missing HX01-RING-01" }
  ],
  "loadTests": [
    { "case": "seismic", "level": 3, "peakUtilisation": 1.28, "failedMembers":
["HX01-STILT-05", "HX01-STILT-06"],
      "members": [ { "memberId": "HX01-STILT-05", "material": "timber",
"utilisation": 1.28 } ] }
  ],
  "score": { "total": 78, "assembly": 34, "safety": 30, "cost": 14 }
}
```

6.3 Storage, integrity and access rules

- **Atomic writes.** Files are written to `*.tmp` and then renamed, so a crash never leaves a half-written session.
- **Versioning.** Every file has a `schemaVersion`. The repository migrates older versions when it loads them and rejects unknown versions without crashing.
- **Single writer.** Only the Data Manager's repositories touch the file system. All other systems receive data objects (section 4.2).
- **Export.** CSV (one row per load test) or the full JSON is written to the cache folder and passed to the Android share sheet. Nothing leaves the device without that user action (NFR-15).
- **Deletion.** Deleting a profile removes its profile entry and its session folder. Settings > Delete all data clears everything except the packaged assets.
- **Volume.** A session file is about 20–60 KB. 50 sessions for each of 10 learners on a shared classroom phone is under 30 MB.

7. Requirements Traceability

Every functional requirement maps to a use case (UC) in section 3.1 and to the class that implements it in section 3.2.

Requirements	Use case	Implementing classes
FR-01 to FR-05	UC-01 Scan and place site	`ARManager`, `InputManager`, `PlacementState`

Requirements	Use case	Implementing classes
FR-06 to FR-08	UC-02 Choose design and site conditions	`UIManager`, `ContentCatalogue`, `SetupState`
FR-09 to FR-14	UC-03 Assemble structure (includes UC-04 Validate build sequence)	`BuildManager`, `TaskManager`, `PlaceMemberCommand`, `MemberFactory`
FR-15 to FR-19	UC-05 Run load test (includes UC-06 Compute utilisation; extended by UC-07 Simulate collapse)	`LoadTestManager`, `StructuralSolver`, `LoadCase` subclasses
FR-20	UC-08 Inspect member	`UIManager`, `StructuralMember`
FR-21	UC-09 Modify and re-test	`BuildManager`, `LoadTestManager`
FR-22	UC-10 View score and feedback	`ScoreManager`, `IScoringPolicy`
FR-23 to FR-25, FR-28	UC-11 Manage profile and history, UC-12 Export report	`DataManager`, `JsonSessionRepository`, `SessionRecord`
FR-26	UC-13 Import scenario	`ScenarioLoader`, `SimulationManager`
FR-27	All interactive use cases	`AudioManager`